Digital Egypt Pioneers Initiative (DEPI) – R5

Graduation Project Documentation

Aman AI

Real-Time Violence Detection & Alert System

Team Members

#	Name	Role
1	Ahmed Mohamed	Team Leader
2	Mahmoud Alaa-Eldin	Team Member
3	Moaz Ahmed	Team Member
4	Doha Mahmoud	Team Member
5	Taghreed Amr	Team Member
6	Malak Yahia	Team Member

October 2026

1. Project Planning & Management

1.1 Project Proposal

Overview

Aman AI is a Computer Vision and Deep Learning system that automatically detects physical violence and altercations in video streams. The model analyzes video frames to recognize aggressive behavior. When violence is detected, the system draws a red bounding box around the active area to highlight the conflict and triggers an automated alert with timestamps.

Problem Statement

Security cameras generate far more footage than human operators can watch continuously. Fights and assaults are often noticed late or only after the fact. An automated system that flags violent behavior as it happens can shorten response time and reduce the monitoring load on security staff.

Objectives

- Develop a deep learning model that classifies video segments as violent or non-violent.
- Localize the area of conflict and highlight it with a red bounding box.
- Generate an automated alert with the timestamp of each detected incident.
- Train and evaluate the model on pre-recorded video datasets and reach high precision in visual threat detection.
- Process video fast enough to be suitable for real-time use.

Scope

In Scope	Out of Scope
Collecting and preprocessing public violence-detection video datasets Developing, training and tuning the action-detection model Red bounding box highlighting on detected violent activity Alert generation with timestamps Testing and evaluation on pre-recorded videos	Deployment on physical CCTV infrastructure Mobile or web dashboard applications Identification of individuals (face recognition) Audio-based detection Production-scale multi-camera management

Expected Outcome

A trained and tested action-detection model, together with a processing pipeline that takes a video as input and outputs an annotated video (red bounding box on violent segments) and an alert log with timestamps.

1.2 Project Plan

Timeline (Gantt Chart)

The plan below is organized in 10 weeks and can be adjusted to match the official DEPI schedule.

Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Planning & requirements	■	■								
Literature review & dataset collection	■	■	■							
Data preprocessing & augmentation			■	■						
Model development & training				■	■	■	■			
Evaluation & fine-tuning						■	■	■		
Detection & alert pipeline							■	■	■	
Testing & documentation								■	■	■
Final presentation										■

Milestones

Milestone	Description	Target
M1	Project proposal and plan approved	Week 1–2
M2	Dataset collected, cleaned and split (train / validation / test)	Week 4
M3	Baseline model trained	Week 6
M4	Final model trained, tuned and evaluated	Week 8
M5	Full pipeline demo (detection + red box + timestamped alerts)	Week 9
M6	Final documentation and presentation delivered	Week 10

Deliverables

- Project proposal and project plan.
- Prepared dataset with documented preprocessing steps.
- Trained violence-detection model (weights and training notebooks / source code).
- Detection pipeline producing annotated video and timestamped alert log.
- Evaluation report (metrics, confusion matrix, error analysis).
- Final documentation and presentation.

Resource Allocation

Resource	Details
Human resources	6 team members, each with a defined role (see 1.3)
Programming & libraries	Python, PyTorch / TensorFlow, OpenCV, NumPy, scikit-learn
Compute	GPU environment such as Google Colab or Kaggle for training
Data	Public violence-detection video datasets (see Section 2.3)

Resource	Details
Collaboration	GitHub for version control; shared drive and chat for coordination

1.3 Task Assignment & Roles

Member	Role	Main Responsibilities
Ahmed Mohamed	Team Leader / Project Manager	Planning, task tracking, communication with supervisors, integration of all components
Mahmoud Alaa-Eldin	Model Development Lead	Model architecture selection, training, hyperparameter tuning
Moaz Ahmed	Computer Vision Engineer	Red bounding box localization, frame processing, alert and timestamp logic
Doha Mahmoud	Data Engineer	Dataset collection, cleaning, labeling checks, augmentation
Taghreed Amr	Testing & QA Engineer	Test cases, model evaluation, false-positive / false-negative analysis
Malak Yahia	Documentation & Presentation Lead	Project documentation, literature review, reports and final presentation

1.4 Risk Assessment & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Limited or unbalanced dataset	Medium	High	Combine several public datasets; use data augmentation and class weighting
False alarms on non-violent activity (sports, playing, hugging)	High	High	Include hard negative examples; tune the decision threshold; require several consecutive violent segments before alerting
Overfitting to the training videos	Medium	High	Cross-dataset validation, dropout, early stopping, augmentation
Low video quality, occlusion, poor lighting	Medium	Medium	Augment with blur, brightness and noise variations
Slow inference that is unsuitable for real-time use	Medium	High	Use a lightweight backbone, reduce frame size, sample frames, optimize the pipeline
Limited GPU resources for training	Medium	Medium	Use Colab / Kaggle GPUs, transfer learning, checkpoints
Schedule delays or unavailable team members	Medium	Medium	Weekly progress meetings, task buffers, backup owner for each task
Privacy and ethical concerns of video data	Low	Medium	Use only public research datasets and do not identify individuals

1.5 KPIs (Key Performance Indicators)

The targets below are the success criteria set by the team for the final model.

KPI	Description	Target
Accuracy	Share of video segments classified correctly	≥ 90%
Precision	Share of raised alerts that are real violence	≥ 85%
Recall	Share of real violent incidents that are detected	≥ 90%
F1-score	Balance between precision and recall	≥ 0.88
False alarm rate	Non-violent segments wrongly flagged as violent	≤ 10%
Processing speed	Frames processed per second on GPU	≥ 15 FPS
Alert delay	Time between start of violence and the alert	≤ 2 seconds
Timestamp accuracy	Difference between alert time and the real event time	≤ 1 second

2. Literature Review

2.1 Background

Automatic violence detection is a branch of human action recognition and video anomaly detection. Unlike image classification, it requires understanding motion over time: a punch and a handshake can look similar in a single frame, but their movement patterns across frames are very different. Research in this area has moved from hand-crafted motion features to deep neural networks that learn spatial and temporal features directly from video.

2.2 Related Approaches

Approach	How it works	Strengths	Limitations
Hand-crafted features	Motion descriptors such as optical flow and HOG feed a classic classifier (e.g., SVM)	Light and fast; simple to train	Weak on complex scenes; needs manual feature design
CNN + LSTM	A CNN extracts per-frame features and an LSTM models the sequence	Captures temporal order; easy to build with pretrained CNNs	Sequential processing is slower; limited motion detail
3D CNNs (C3D, I3D)	3D convolutions learn spatial and temporal features together	Strong motion modeling	Heavier; needs more data and compute
Two-stream networks	One stream for RGB frames, one for optical flow	High accuracy on action recognition	Optical flow is costly to compute in real time
Pose / skeleton-based	Human pose keypoints are tracked and classified	Focuses on body movement; less affected by background	Fails in crowds and with occlusion
Video transformers	Attention across space and time (e.g., TimeSformer, Video Swin)	State-of-the-art accuracy	Large models; high compute cost
Object-detection-based (YOLO family)	Detects the region of the action and classifies it	Gives a bounding box directly; fast	Needs box-level annotations or a separate temporal component

2.3 Datasets Used in the Field

Dataset	Description
Hockey Fight	About 1,000 clips from ice hockey games, balanced between fight and non-fight
Movies Fight	About 200 clips of fights and non-fights taken from movies
Violent Flows	Short crowd-violence videos collected from the web
RWF-2000	About 2,000 real surveillance clips of fights and non-fights
Real Life Violence Situations (RLVS)	About 2,000 videos of real street fights and non-violent activities
UCF-Crime	Long, untrimmed surveillance videos covering 13 types of anomalies, including fighting and assault

Note: *dataset sizes are approximate and should be confirmed from each dataset's official source before the final submission.*

2.4 Gap Analysis & Our Approach

Many published systems focus only on clip-level classification (violent or not) and do not show where the violence happens or when to alert. Aman AI addresses this by combining three elements: action detection with a deep learning model, visual localization through a red bounding box, and timestamped automated alerts. The model is developed and evaluated on pre-recorded video datasets to reach high precision in visual threat detection.

2.5 Feedback & Evaluation

Lecturer's assessment of the project (to be completed after the project review).

Review Date	Reviewer	Feedback	Action Taken

2.6 Suggested Improvements

- Train on a larger and more diverse dataset to improve generalization.
- Add pose estimation or audio cues to reduce false alarms.
- Connect the system to live camera streams (RTSP) and multiple cameras.
- Send alerts by SMS, e-mail or a mobile notification.
- Optimize the model for edge devices (quantization, TensorRT, ONNX).
- Extend detection to other threats such as weapons or crowd panic.
- Build a web dashboard for reviewing incidents and alert history.

2.7 Final Grading Criteria

Breakdown of marks based on documentation, implementation, testing, and presentation. Marks are to be filled in by the lecturer.

Criterion	What is assessed	Marks
Documentation	Completeness, clarity and structure of project documents	
Implementation	Model quality, detection pipeline, red-box highlighting, alerts	
Testing	Test coverage, evaluation metrics, error analysis	
Presentation	Delivery, demo, and answers to questions	
Total		

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Role	Needs & Expectations
Security operators	Primary users who monitor cameras	Fast and accurate alerts; few false alarms; clear visual highlighting of the incident
Facility management (schools, malls, public places)	Decision makers and system owners	Safer premises; reliable system; quick incident review
Emergency responders / police	Receive and act on alerts	Timely and trustworthy alerts with the exact time of the incident
Project supervisors / lecturers	Evaluate the project	A well-documented, correctly evaluated and working model
Development team	Build and maintain the system	Clear requirements; reproducible training and testing
People in monitored areas	Indirect stakeholders	Privacy respected; no identification of individuals

3.2 User Stories & Use Cases

User Stories

ID	As a…	I want to…	So that…
US-01	Security operator	upload or stream a video into the system	it is analyzed automatically for violence
US-02	Security operator	see a red box around the area of violent activity	I can immediately see where the incident is
US-03	Security operator	receive an alert with a timestamp	I know exactly when the incident started
US-04	Supervisor	review the list of detected incidents	I can verify the system's decisions
US-05	Developer	run the model on a test dataset and get metrics	I can measure and improve its performance
US-06	Facility manager	have fewer false alarms	staff trust the alerts and respond to them

Use Cases

UC-01: Detect Violence in Video

Actor	Security operator
Precondition	A video file or stream is available to the system
Main flow	1. The operator provides the video. 2. The system extracts and preprocesses frames. 3. The model classifies each segment as violent or non-violent. 4. For violent segments, the system draws a red bounding box on the video.
Postcondition	An annotated video is produced
Alternative flow	If the video is unreadable, the system shows an error message

UC-02: Generate Alert

Actor	System (receiver: security operator)
Precondition	Violence was detected with confidence above the threshold
Main flow	1. The system records the start and end timestamps. 2. The system generates an alert containing the timestamp and confidence. 3. The alert is shown to the operator and saved in the log.
Postcondition	The incident is stored in the alert log

UC-03: Review Incident Log

Actor	Supervisor / security operator
Precondition	At least one incident has been logged
Main flow	1. The user opens the alert log. 2. The user selects an incident. 3. The system shows the annotated video segment and its timestamp.
Postcondition	The user confirms or rejects the detection

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall accept pre-recorded video files as input.	High
FR-02	The system shall extract frames and apply preprocessing (resize, normalization).	High
FR-03	The system shall classify video segments as violent or non-violent.	High
FR-04	The system shall draw a red bounding box around the active area of violence.	High
FR-05	The system shall trigger an automated alert when violence is detected.	High
FR-06	Each alert shall include the timestamp of the incident.	High
FR-07	The system shall save the annotated output video.	Medium
FR-08	The system shall keep a log of all detected incidents.	Medium
FR-09	The system shall allow adjusting the detection confidence threshold.	Medium
FR-10	The system shall report evaluation metrics (accuracy, precision, recall, F1) on a test set.	High

3.4 Non-functional Requirements

Category	Requirement
Performance	Process video fast enough for real-time use (target ≥ 15 FPS on GPU) with an alert delay of no more than 2 seconds; high precision and recall as defined in the KPIs.
Security	Video data is stored locally and only accessible to authorized users; no sharing of footage with third parties; only public research datasets are used for training.
Usability	Clear visual output (red box, readable timestamps); simple input and output flow; documented usage steps.
Reliability	The system handles corrupted or unsupported files without crashing; results are reproducible with fixed seeds and saved model weights.
Maintainability	Modular code (preprocessing, model, post-processing, alerts) with comments and version control.
Scalability	The pipeline can later be extended to more videos or camera streams without changing the model design.